STANFORD CS336 / SPRING 2026

Lecture 7

Parallelism：跨 GPU 的通信与训练并行

从 collective primitives 到 Data / Tensor / Pipeline Parallelism 的结构化学习笔记

一句话主线
单 GPU 优化的是 HBM 与片上存储之间的数据移动；多 GPU
优化的是训练状态、数据与激活值在不同设备之间的复制和分片。

讲义主线：collective operations -> hardware topology -> NCCL/PyTorch -> bandwidth benchmark -> DP / TP / PP

本笔记以 lecture_07.py 和课程配图为基础，补充标准性能模型与工程判断。

学习笔记生成时间：2026-10-05 | 输出目录：output/pdf/

目录

编号	主题	你会学到什么
01	课程定位：为什么需要多 GPU 并行	计算、显存、通信的统一视角
02	通信层级：从 L1 到 Ethernet	带宽与延迟为什么决定切分策略
03	集合操作：分布式编程的原语	broadcast / reduce-scatter / all-reduce / all-to-all
04	NCCL 与 torch.distributed	从 API 到硬件 packet
05	通信带宽的测量与公式	all-reduce 与 reduce-scatter 的 benchmark
06	Data Parallelism	复制模型、切 data、同步梯度
07	Tensor Parallelism	切宽、频繁通信、NVLink 友好
08	Pipeline Parallelism	切层、micro-batch、pipeline bubble
09	三种策略的系统比较	sharding axis / communication / bottleneck
10	FSDP / ZeRO：把 all-reduce 拆开	参数、梯度、optimizer state 分片
11	总结、检查题与工程 checklist	面试与作业中的高频问法

阅读方法：每个并行策略都先问两个问题：**沿哪个轴切？** 以及 **切完之后通信什么？** 只要这两个问题回答清楚，DP / TP / PP 就不会混在一起。

01 为什么需要多 GPU 并行

第七讲把视角从“单个 GPU 内部如何少读少写”，推进到“多个 GPU 之间如何少传、怎样切分”。两个动机同时存在：模型或训练状态放不进单卡显存；以及希望用更多算力缩短训练时间。

1.1 单卡优化的三个层次

- **Fusion / tiling**：把多次内存读写合并为更少的 kernel 与更规则的 tile，降低 HBM traffic。
- **Recomputation**：少存激活值，反向时重算，用 compute 换 memory。
- **Low precision**：用 bf16 / fp16 / fp8 减少字节数，同时控制数值误差。

第七讲对应的是更高一层：**replication / sharding**。同一组参数可以每个 rank 复制一份，也可以切成多片；同一批数据可以切给不同 rank，也可以让每个 rank 都处理完整 batch。切分轴决定了通信模式和硬件要求。

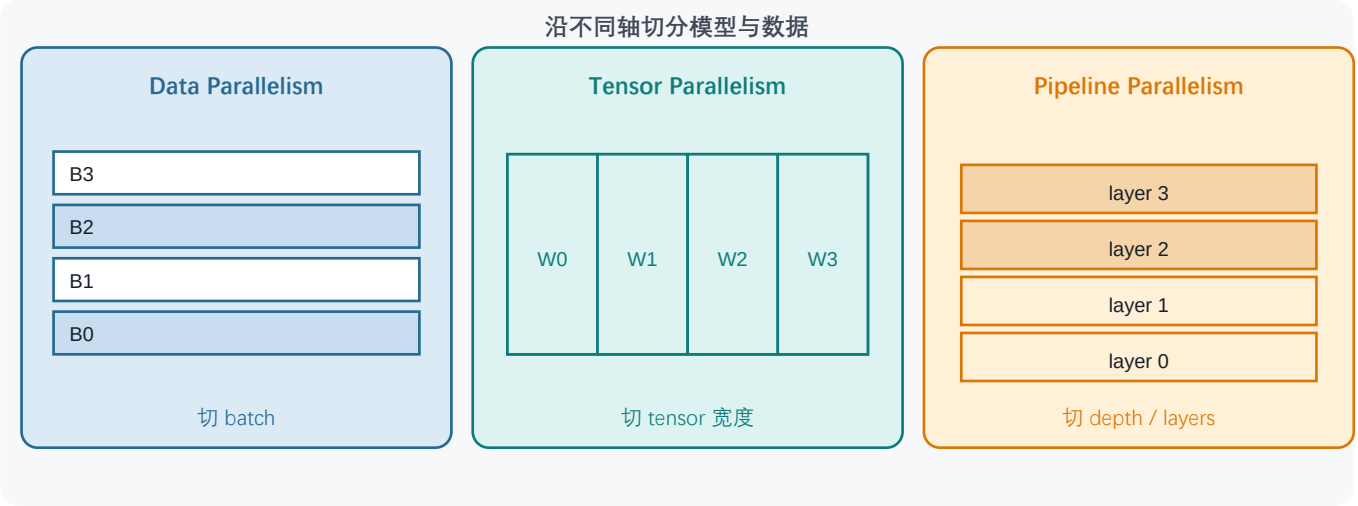


图 1 三种最常见的切分轴：batch、tensor width、depth/layers

1.2 训练状态的显存账本

对参数量 P 的模型，AdamW 混合精度训练还要保存参数副本、梯度、fp32 master weights、一阶/二阶动量以及激活值。即使忽略 activation，模型状态也常常是 P 的十几倍字节数。

> 粗略账本：bf16 参数 2P + bf16 梯度 2P + fp32 master 4P + Adam m/v 8P = 约 16P bytes（不含 activation）。

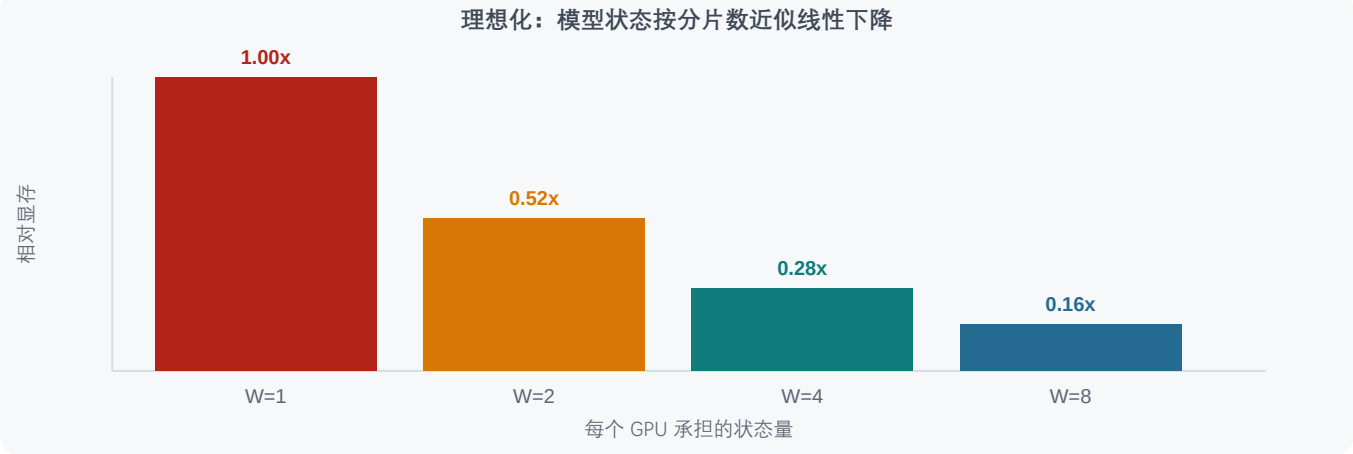


图 2 分片越多，每张卡承担的状态量越少；横轴是理想化模型，真实系统还要加 communication buffer

关键判断：并行不是为了“平均使用所有 GPU”，而是为了让通信成本落在足够快的互连上。数据并行、张量并行、流水线并行的差别，本质上是**通信频率、消息大小和带宽要求**的差别。

02 通信层级：从 L1 到 Ethernet

现代训练集群是一个分层系统。越靠近计算单元，带宽越高、延迟越低；越往外走，容量和可扩展性更强，但每一字节的代价更高。

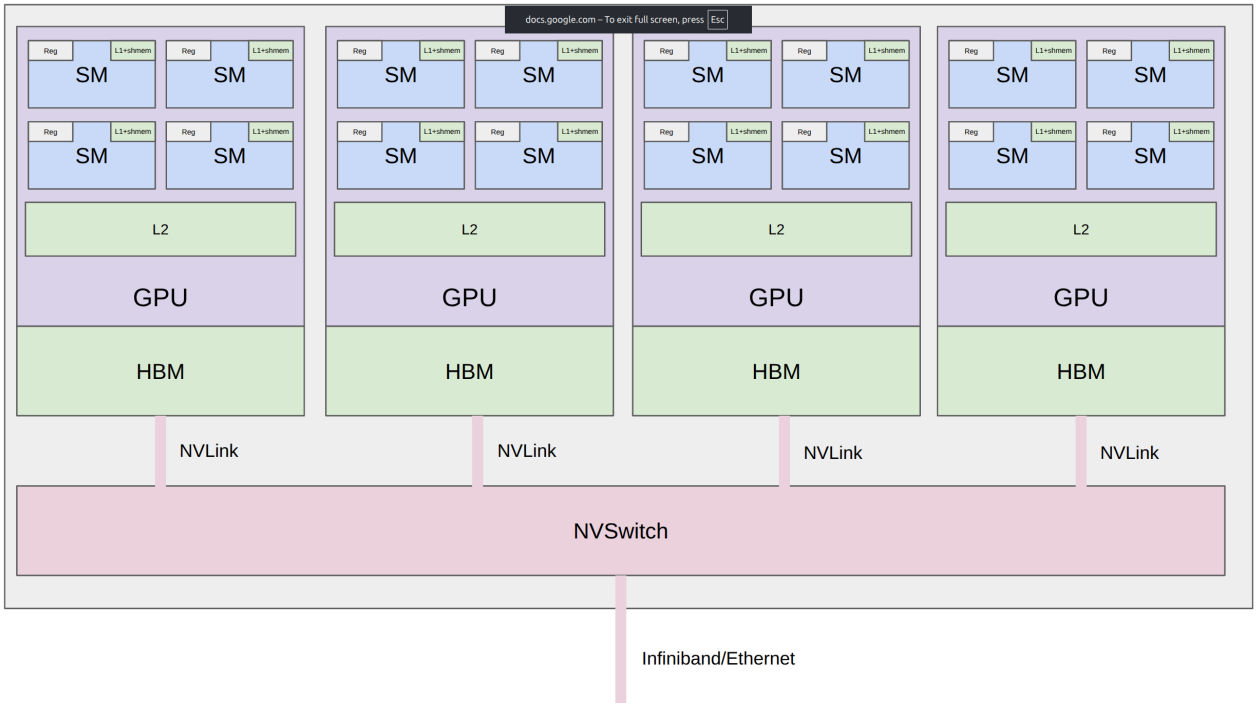


图 3 多 GPU 节点的典型结构：GPU 内部有 SM / L2 / HBM，节点内通过 NVLink/NVSwitch，节点间通过 InfiniBand 或 Ethernet

层级	典型位置	带宽量级（讲义口径）	对并行策略的含义
L1 / shared memory	SM 片上	最高、容量最小	tiling、fusion 的主战场
HBM	单 GPU	约 8 TB/s（B200 口径）	单卡 kernel 的带宽上限
NVLink / NVSwitch	节点内 8 GPU	约 1.8 TB/s（B200 NVLink 5.0）	TP 至少应优先放这里
InfiniBand	多节点 pod	约 0.05 TB/s	DP / PP / FSDP 的大消息通信
Ethernet	跨 pod / datacenter	约 200 MB/s 的经典口径	最慢，常需要 RDMA / RoCE

2.1 延迟和带宽要同时算

一次通信的时间可以先近似为：T = latency + bytes / bandwidth。小消息被 latency 支配，大消息被带宽支配。因此 TP 更怕频繁的小 all-gather；PP 的 stage boundary 消息可以较大，所以对网络带宽的要求通常没有 TP 那么苛刻。

2.2 为什么节点边界这么重要

- 节点内 NVLink 是 GPU 到 GPU 的高带宽路径，适合每层都通信的 TP。
- 跨节点 InfiniBand 仍比 NVLink 慢一个数量级，适合带宽压力较小的 PP 或参数分片通信。
- Ethernet 经典路径需要经过 CPU 协议栈；RDMA 让 GPU 直接读写远端显存，绕过 CPU。

- RoCE 把 RDMA 能力带到 Ethernet，是成本与性能之间的折中。

> **工程结论**：先画拓扑，再决定切分。把高频、小消息的通信放在 NVLink 域内；把低频、大消息的通信放到跨节点网络上，通常比单纯增加 GPU 数量更有效。

03 集合操作：分布式编程的原语

Collective operation 是一组设备共同执行、具有固定通信模式的集合通信操作。NCCL 把它们翻译为底层 packet；PyTorch 把它们暴露为 torch.distributed API。

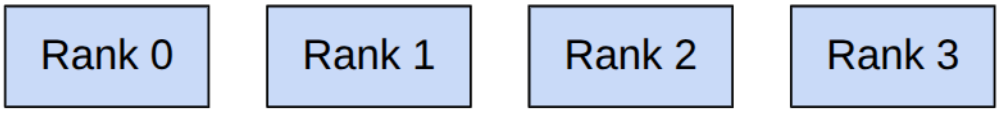


图 4 rank 是设备的编号，world size 是设备总数；后续示例默认 world size = 4



图 5 集合操作速查：先看数据从哪里来、到哪里去，再看是否包含 reduce

操作	输入 -> 输出	典型语义	训练中的用途
Broadcast	0 -> all	复制同一份数据	初始化 checkpoint / 超参数
Scatter	0 -> each	分片发送	理解 reduce-scatter 的基础
Gather	each -> 0	收集到 rank 0	汇聚指标 / 诊断
Reduce	each -> 0	sum/min/max	理解 all-reduce 的基础
All-gather	each -> all	每人贡献一块，所有人拿全量	TP 恢复完整激活；FSDP 取参数
Reduce-scatter	each -> each	reduce 后按块分发	梯度求和并分片存储
All-reduce	each -> all	reduce 后所有人拿全量	DDP 同步梯度
All-to-all	every pair	最通用的交换	MoE token routing

3.1 用矩阵理解 reduce-scatter

假设 world size = 4，每个 rank 都持有一个长度为 4 的向量，reduce-scatter 会先对每一列求和，再把 4 个结果分给 4 个 rank。

Rank 0		`[0, 1, 2, 3]`
Rank 1		`[1, 2, 3, 4]`
Rank 2		`[2, 3, 4, 5]`
Rank 3		`[3, 4, 5, 6]`
reduce-scatter 结果		`[6] [10] [14] [18]`

如果再把每个 rank 得到的块 all-gather 回所有 rank，结果就是 all-reduce。所以讲义给出的最重要恒等式是：

$$\text{All-reduce} = \text{Reduce-scatter} + \text{All-gather}$$

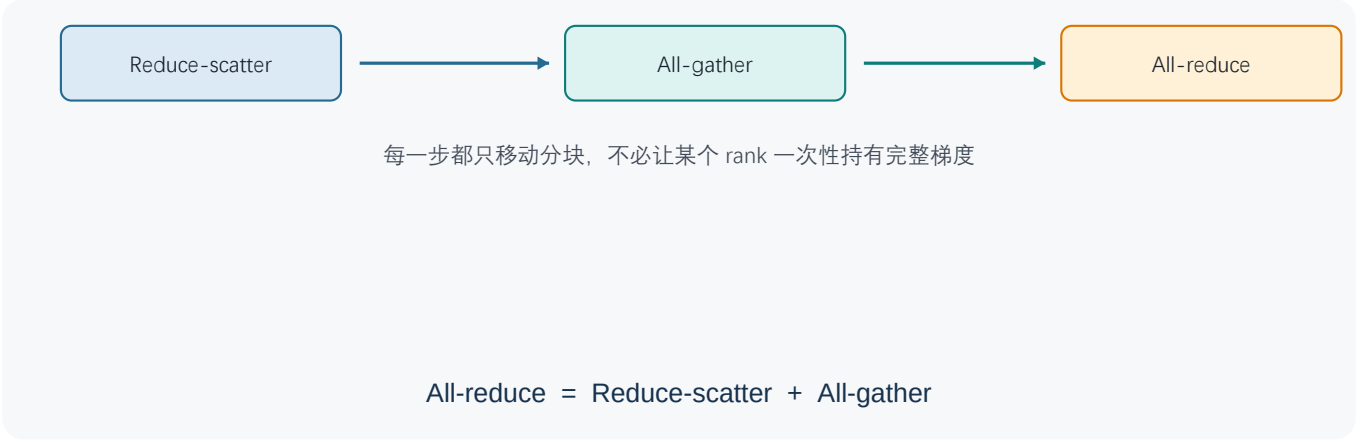


图 6 all-reduce 不一定要一次完成，拆分后在内存和复制策略上更灵活

3.2 all-to-all 与 MoE

MoE 中每个 token 会被 router 分到不同 expert，expert 又分布在不同 rank。All-to-all 允许每个 rank 把 token 发给目标专家，再接收回应。当各专家负载均衡时，它看起来像一个矩阵转置；负载不均衡时，某些 rank 会成为长尾。

> **记忆法**：reduce 是“先做运算”，scatter 是“把块分开”，gather 是“把块合起来”，**all** 表示最终所有 rank 都拿到；all-reduce 就是 reduce 后所有人拿全量。

04 NCCL 与 torch.distributed

NCCL 是 NVIDIA 的集合通信库。它不只是“发 packet”，还会根据拓扑选择 ring、tree 等算法，并启动 kernel 完成数据和同步。

NCCL 做的事	为什么要关心
探测拓扑	节点数、NVLink/PCIe、switch、网卡决定可用路径
优化路径	同一个 collective 在不同拓扑上可能使用不同 algorithm
启动 CUDA kernel	通信本身也要占 GPU 时间，并可能和计算竞争 SM

4.1 PyTorch API 的阅读顺序

读分布式代码时先看 process group，再看 collective 的 input/output shape，最后看 async 与 overlap 的位置。

```
import torch.distributed as dist

# DDP: gradient all-reduce
dist.all_reduce(param.grad, op=dist.ReduceOp.AVG)

# FSDP / ZeRO style: ■■■■■■■ all-gather
dist.all_gather_into_tensor(output_tensor, input_tensor)

# gradient reduce + sharded storage
dist.reduce_scatter_tensor(output, input, op=dist.ReduceOp.SUM)

# pipeline stage boundary
dist.send(tensor=activation, dst=rank + 1)
dist.recv(tensor=activation, src=rank - 1)
```

4.2 讲义示例与真实训练的差别

- 讲义为了让算法结构清楚，很多 collective 使用同步调用，没有展开 communication/computation overlap。
- 真实 DDP/FSDP 会把 all-reduce、all-gather、reduce-scatter 分块，并尽量塞进 backward 的剩余时间里。
- PyTorch 的高层封装（DDP、FSDP）仍然建立在同一批 collective primitive 上。

看代码时的三连问:
1. 通信对象是什么？参数、梯度还是 activation?
2. 通信发生在哪一层？每层、每 step，还是只在 stage boundary?
3. 通信能否与计算 overlap？如果不能，瓶颈往往先出现在这里。

05 通信带宽的测量与公式

Benchmark 的目的不是记住某个 GB/s 数字，而是理解：你测到的有效带宽取决于消息大小、world size、拓扑和算法。讲义用 all-reduce 与 reduce-scatter 展示了如何把 wall-clock time 转成有效带宽。

5.1 All-reduce benchmark

设单个 rank 的输入大小为 S bytes，world size 为 W。讲义把发送量记为 S × 2 × (W - 1)，其中 2 代表 send + receive，(W - 1) 代表通信轮数；再把所有 rank 的 duration 汇总，得到聚合有效带宽。

$$\text{sent_bytes} = \text{size_bytes} \times 2 \times (W - 1)$$
$$\text{total_duration} = W \times \text{duration}$$
$$\text{effective_bandwidth} = \text{sent_bytes} / \text{total_duration}$$

讲义进一步指出，对大消息有近似 effective_bandwidth ≈ 2 × size_bytes / duration，这个结果对 world size 和 ring/tree 拓扑不敏感。

5.2 Reduce-scatter benchmark

reduce-scatter 不需要像 all-reduce 那样再广播完整结果，因此讲义使用 sent_bytes = data_bytes × (W - 1)。all-reduce 的总移动量约是 reduce-scatter 的 2 倍，时间也约为 2 倍，所以两者的有效带宽看起来相似。

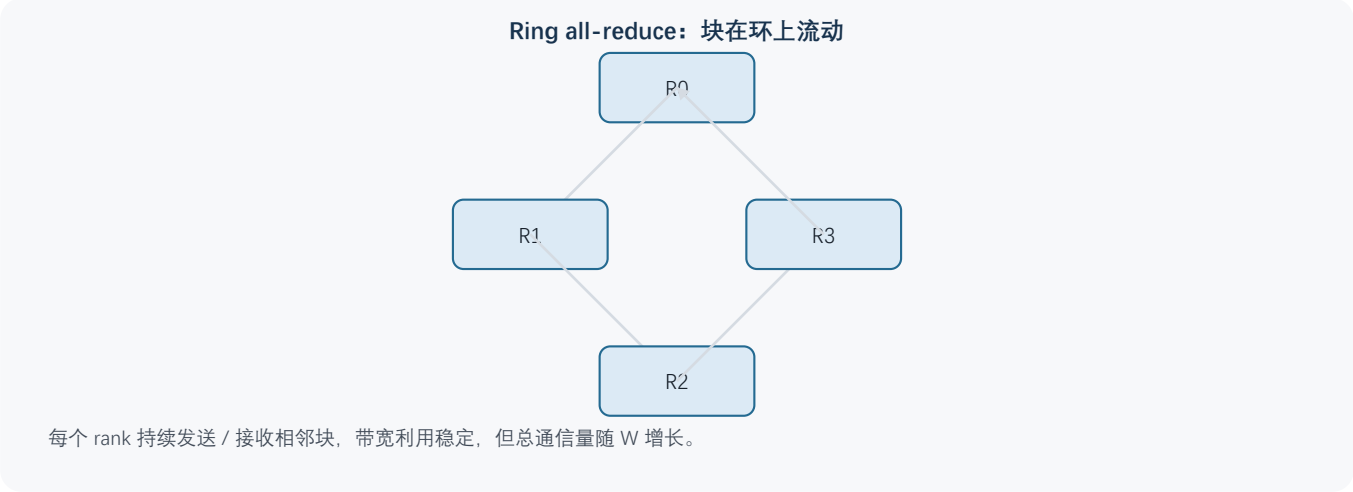


图 7 把 collective 想成数据块在拓扑上流动，而不是“一个函数调用瞬间完成”

不要误读公式：这里的 sent_bytes 是讲义用于估计聚合有效带宽的 aggregate 计数，不是某个 GPU 网卡在每一纳秒实际传输的线速。工程上真正要测的是：目标消息大小下，端到端 step time 是否下降。

06 Data Parallelism：复制模型，切 batch

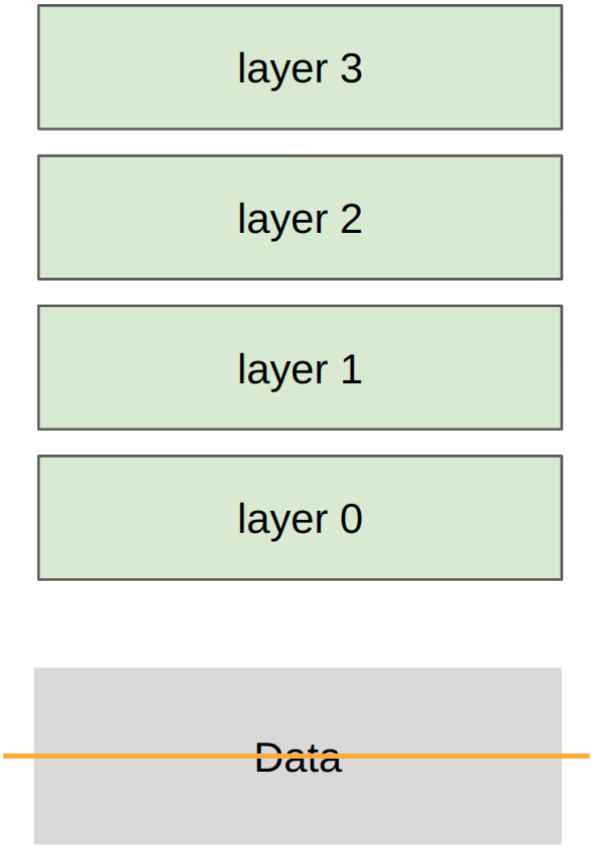


图 8 数据并行：数据沿 batch 维切开，每个 rank 有完整模型副本

DP 的核心操作非常干净：每个 rank 用自己的 local batch 做完整 forward/backward，得到 local gradient；然后用 all-reduce 求平均，使所有 rank 的 gradient 一致，最后各自执行相同的 optimizer step。

$$\text{global_batch} = W \times \text{local_batch}$$
$$g_i = \text{gradient on rank } i$$
$$g_{sync} = (1/W) \times \Sigma_i g_i$$
$$\theta \leftarrow \theta - \eta \times g_{sync}$$

6.1 为什么参数会保持同步

- 初始参数相同：所有 rank 从同一 checkpoint 开始。
- 每步 gradient 相同：all-reduce 的 AVG 让每个 rank 拿到同一个同步梯度。
- optimizer 更新相同：相同 θ 和相同 gradient 产生相同 θ。

因此 DP 的 loss 在每个 rank 上可以不同，但参数在每次 step 后应当一致。如果参数发生 drift，通常说明有随机性、buffer 没同步，或者某些层被排除在 DDP 之外。

6.2 通信代价与瓶颈

- **通信频率**：每个 training step 后同步一次完整 gradient。
- **消息大小**：约等于模型参数量，通常是大消息。
- **带宽需求**：跨节点 all-reduce，对网络带宽敏感，但对每层 latency 的敏感度低于 TP。
- **显存节省**：不节省模型状态；每张卡仍要保存完整参数、梯度和 optimizer state。
- **主要优势**：实现成熟、扩展性好、计算和通信容易 overlap。

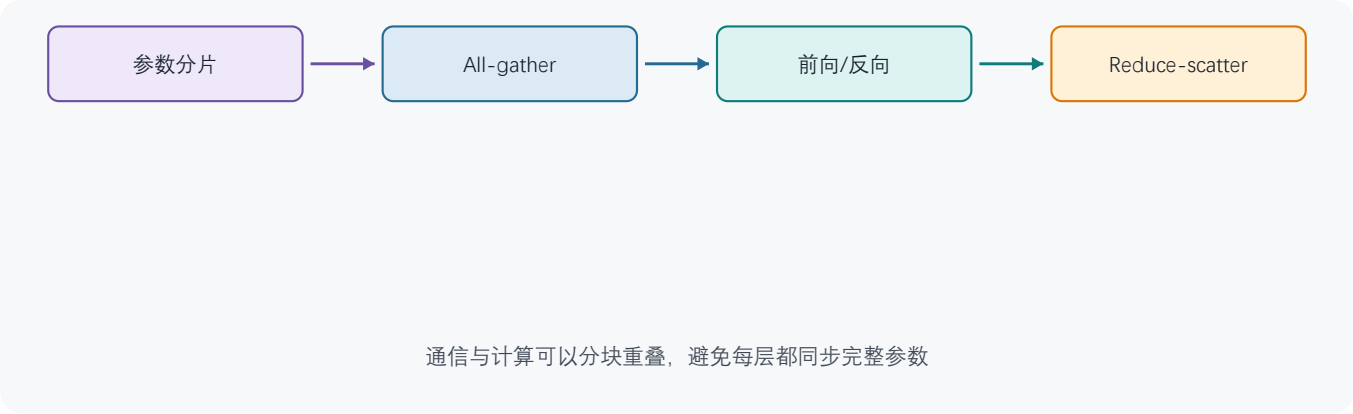


图 9 当模型状态本身放不下时，下一步就是把 all-reduce 拆成 all-gather + reduce-scatter

6.3 DDP、FSDP 与 ZeRO 的关系

方案	每张卡保存什么	通信模式	解决的问题
DDP	完整参数 + 完整梯度 + optimizer state	gradient all-reduce	提高吞吐；不解决单卡模型状态容量
FSDP	参数、梯度、optimizer state 分片	all-gather + reduce-scatter	同时降低模型状态显存
ZeRO	按 stage 分片 optimizer / gradient / parameter	类似 all-gather / reduce-scatter	把 DP 的内存优化分阶段推进

从系统视角看，DP 只把“数据”切了，没有把“模型状态”切掉。当模型参数、梯度和 Adam 状态加起来超过单卡容量时，DP 的副本策略会失效，这时需要用 FSDP/ZeRO 的空闲通信换显存。

DP 的适用场景：模型能放进单卡，batch 足够大，集群跨节点但每步通信可 overlap。如果模型放不下，DP 不是“稍慢一点”，而是根本无法启动。

07 Tensor Parallelism：切宽度，换通信

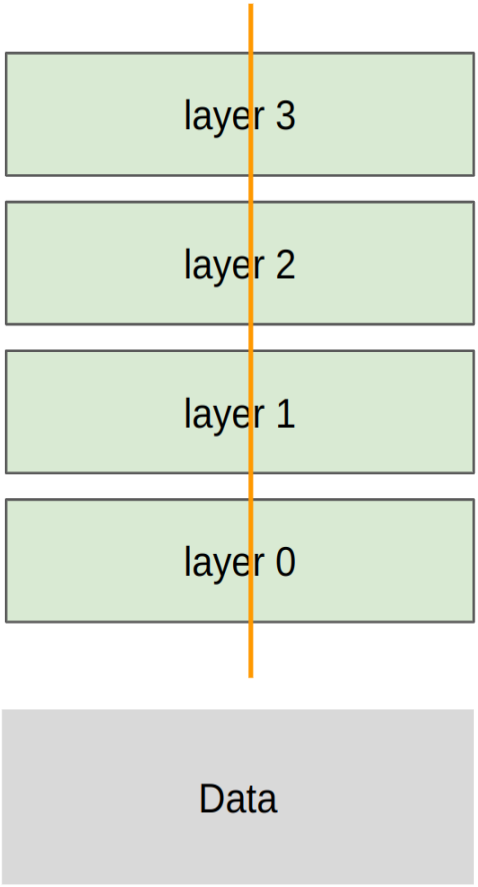


图 10 张量并行：每一层都沿 width / hidden dimension 切分，每个 rank 持有一部分权重

TP 把矩阵乘法中的维度切开：每个 rank 只计算一部分输出，再通过 all-gather、all-reduce 或 reduce-scatter 组合成下一层需要的 activation。它直接减少单卡参数和 activation，但引入了非常频繁的通信。

7.1 最小例子：Y = GELU(XW)

假设 hidden dimension 为 d，world size 为 W。把权重矩阵 W 按输出维度切成 W 份：W = [W_0, W_1, ..., W_{W-1}]。每个 rank 计算 XW_i，得到 batch × (d/W) 的局部激活。如果下一层需要完整 hidden states，就 all-gather 回 batch × d。

Forward: Y_i = GELU(X × W_i)
Communication: All-gather(Y_0, ..., Y_{W-1})
Local parameter: P / W

7.2 真实 Transformer 中的 TP

- **Column parallel**：按输出维度切权重，每个 rank 产生一部分激活。
- **Row parallel**：按输入维度切权重，每个 rank 得到 partial sum，再用 all-reduce 合并。
- 讲义用每层 all-gather 作为最小实现；Megatron 类系统会安排 column / row parallel，让通信尽量只出现在层边界。
- TP 通常放在 NVLink 域内，而不是跨节点：因为每层都要通信，latency 和带宽都关键。

7.3 为什么 TP 对硬件最挑剔

维度	TP 的要求	原因
通信频率	每层甚至每半个 matmul	消息多、依赖链长
延迟	越低越好	频繁 barrier / collective 会放大 latency
带宽	NVLink/NVSwitch 级	activation 和 partial sum 反复传输
扩展范围	通常 2/4/8 的节点内小组	跨节点后通信成本迅速超过计算收益

TP 的核心权衡：它省的是每张卡的参数与 activation，代价是每层都可能通信。当 NVLink 足够快时，TP 能让单卡装不下的层继续训练；当通信跨节点时，TP 往往不再划算。

7.4 TP / DP 的典型组合

大规模训练不会只选一种并行。一个常见的组合是：节点内做 TP，节点间做 DP 或 PP；有时再叠加 sequence parallel、expert parallel。组合的排序原则是让通信频率最高的策略使用最快的互连。

TP group 放在 node 内 -> 高频 collective 落在 NVLink
DP / PP group 跨 node -> 低频或大消息通信落在 InfiniBand
理论上也可用 hybrid sharding / HSDP 进一步分层

从微观看，TP 让每个 rank 只计算一个窄矩阵，单卡算术强度可能下降，因为同样的 activation 通信量分摊到更小的 matmul 上。因此 TP 不只是“通信贵”，还会改变 kernel 的 arithmetic intensity。

面试答法：TP 切的是 width，通常每层通信，依赖 NVLink；它的收益是把参数和 activation 摊到多张卡，成本是高频 collective 和更小的局部矩阵。

08 Pipeline Parallelism：切层，填充流水线

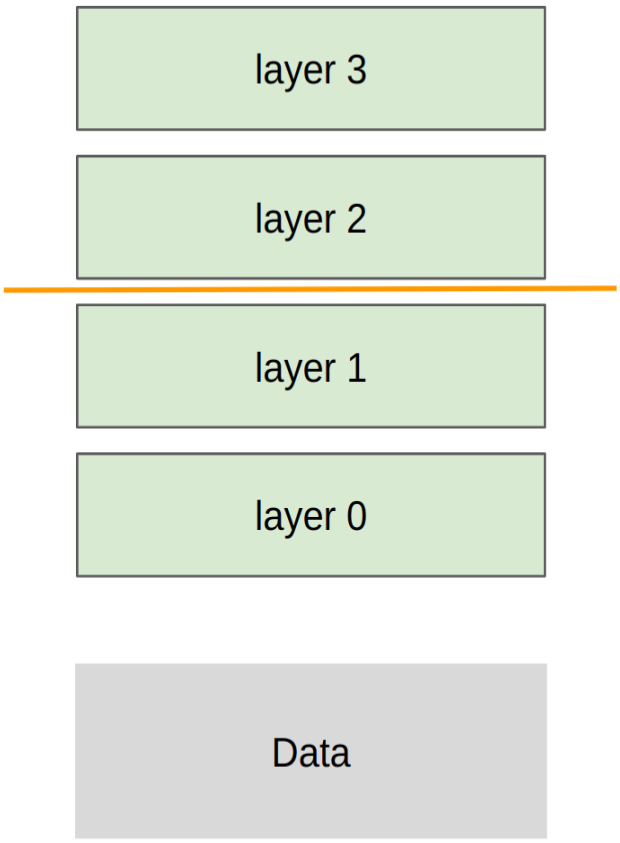


图 11 流水线并行：按 depth / layer 把模型分给不同 rank，activation 在 stage 之间传递

PP 把模型按层切成若干 stage。每个 rank 只保存部分层，forward 时把 activation 发给下一 stage，backward 时把 gradient 发回上一 stage。因为只在 stage boundary 通信，PP 通常比 TP 更能容忍跨节点网络。

8.1 Micro-batch 解决什么？

如果一次只送一个完整 batch，后面的 stage 会一直等待前面的 stage，GPU 在大部分时间空闲。Micro-batch 把 batch 切成更小的块，让不同 stage 在同一时间处理不同的 chunk，像工厂流水线一样重叠起来。

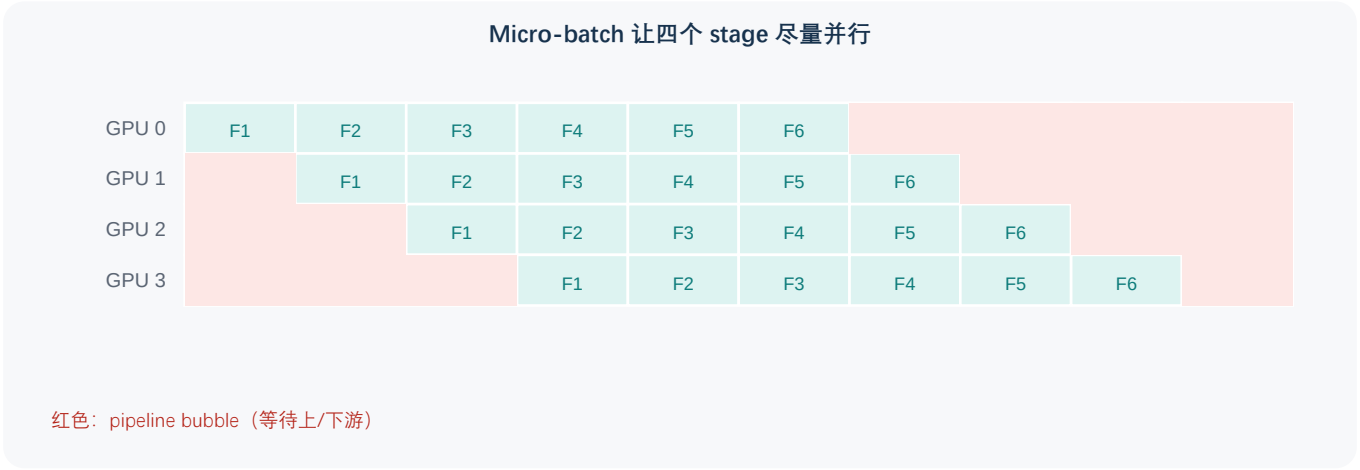


图 12 简化的多 stage 流水线：F1/F2 表示不同 micro-batch 的前向；真实系统还会穿插反向

8.2 流水线气泡近似

设 stage 数为 P，micro-batch 数为 M。最简单的 schedule 中，启动和排空的空洞占比近似为 (P - 1) / (M + P - 1)。M 太小会形成明显 bubble；M 太大则每个 micro-batch 太小，matmul 效率和内存行为可能变差。

bubble_fraction ≈ (P - 1) / (M + P - 1)
例：P = 4, M = 8 -> 3 / 11 ≈ 27% 的空洞
例：P = 4, M = 32 -> 3 / 35 ≈ 8.6%

8.3 为什么 PP 对网络相对友好

- 通信次数较少：只在 stage boundary 发生。
- 消息通常较大：一个 micro-batch 的 activation 可以整块发送。
- 可以使用 send/recv 点对点通信，不一定需要全局 collective。
- 但需要处理 bubble、activation 保存和反向阶段的调度。

PP 的核心权衡：它省模型状态，也允许跨节点扩展；代价是空泡、调度复杂，以及 micro-batch 太小带来的效率损失。优化方向通常是 1F1B、interleaved schedule 和更好的 overlap。

8.4 GPipe 与 1F1B

调度	思路	内存代价	空泡
GPipe	先做多个 micro-batch 的前向，再统一做反向	要同时保存较多 activation	较明显
1F1B	每个 stage 前向一个、反向一个，穿插执行	activation 峰值更低	更小且更平滑
Interleaved	每个 rank 负责多段 layer，进一步打散调度	调度更复杂	可继续降低

“切割 depth”看起来简单，但真正的难点是调度：前向计算与反向计算必须按依赖关系排列，同时尽量让每个 stage 都有活干。PP 的工程价值在于它把模型层分布到多张卡，同时把通信限制在少数 stage boundary。

诊断 PP 性能时先看三件事：micro-batch 数是否足够、stage 是否负载均衡、communication 是否真的与 compute overlap。只要 stage 负载偏斜，公式再漂亮也会被最慢 stage 卡住。

09 三种策略的系统比较

DP、TP、PP 不是三个互斥的选项，而是三种切分轴的组合。下表把每个策略放回它最自然的硬件位置。

策略	切割轴	每张卡保存什么	主要通信	带宽 / 延迟	主要瓶颈
DP	batch	完整模型状态副本	gradient all-reduce	大消息；跨节点带宽敏感	模型状态容量；同步开销
TP	width / hidden	每个 tensor 的一部分	per-layer all-gather / all-reduce	高频；NVLink 级带宽与延迟	通信频率与局部矩阵效率
PP	depth / layers	部分层与 activation	send / recv activation	低频大消息；可跨节点	pipeline bubble 与负载均衡
FSDP/ZeRO	参数 / 梯度 / optimizer state	分片的模型状态	all-gather + reduce-scatter	大消息；可与计算 overlap	通信与显存节省之间的权衡

9.1 一张决策图

如果单卡放不下模型状态：优先考虑 PP / FSDP / TP 的组合。
如果节点内有多张 GPU，且层内矩阵很大：用 TP 切开层。
如果跨节点带宽有限但模型能分层：用 PP 切开 depth。
如果模型能放下，只想提高吞吐：用 DP，并尽量 overlap gradient all-reduce。

9.2 组合的硬件直觉

把最快互连留给最频繁通信：
节点内：TP（高频 collective）
节点间：PP / DP（低频或大消息）
更多副本：DP + FSDP / HSDP（进一步把状态摊开）

10 FSDP / ZeRO：把 all-reduce 拆开

FSDP 与 ZeRO
不是另一个神秘的并行类别，而是把数据并行的“所有卡都保存完整状态”改成“参数、梯度、optimizer state 分片存储”，再用 collective 按需取回或写回。

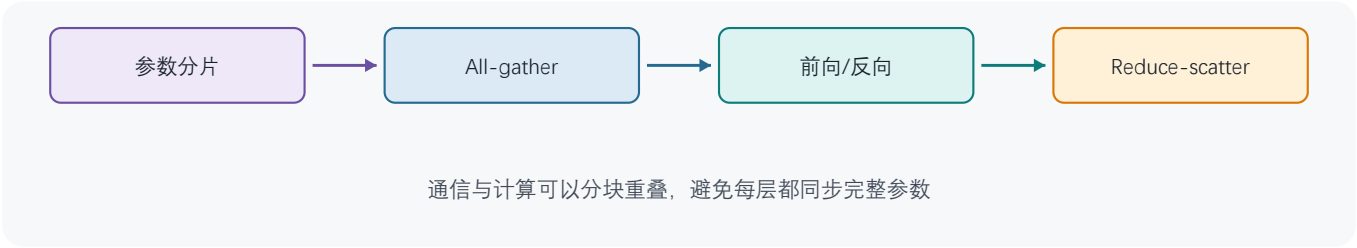


图 13 FSDP/ZeRO 的核心循环：all-gather 取回参数，计算，再用 reduce-scatter 写回分片梯度

10.1 三个分片对象

对象	不分片时	分片后	通信
optimizer state	每卡 Adam m/v	每个 rank 只保存 1/W	update 后按需同步参数
gradient	每卡完整梯度	reduce-scatter 后按块保存	backward 结束 reduce-scatter
parameter	每卡完整参数	all-gather 后按层临时恢复	forward 前 all-gather

显存收益可以用模型状态近似表示：每个 GPU 从约 16P bytes 降到约 16P/W bytes；实际还会受到 bucket、buffer、临时 all-gather buffer 和 activation 的影响。

本质对比：DDP 省的是时间，不省模型状态；FSDP/ZeRO 省的是模型状态，代价是引入 all-gather / reduce-scatter，并把通信塞进 forward 与 backward 的关键路径。

11 总结、检查题与工程 checklist

11.1 必须记住的五句话

- 多 GPU 并行有两个目标：容量（放得下）和速度（算得快）。
- 通信时间近似由 latency + bytes / bandwidth 决定，小消息看 latency，大消息看 bandwidth。
- DP 切 batch，复制模型；TP 切 width，频繁通信；PP 切 depth，承担 pipeline bubble。
- All-reduce = reduce-scatter + all-gather；这条恒等式是 DDP、FSDP、ZeRO 的共同骨架。
- 并行策略要和硬件拓扑匹配：高频集体通信尽量放 NVLink 域，跨节点尽量放低频或大消息通信。

11.2 自测题

- 为什么 data parallelism 的 loss 可以每张卡不同，但参数仍然同步？
- 为什么 tensor parallelism 对 NVLink 的依赖比 pipeline parallelism 更强？
- Reduce-scatter 与 all-gather 如何组合成 all-reduce？每一步的输出 shape 是什么？
- 如果 pipeline 有 4 个 stage、16 个 micro-batch，最简单的 bubble fraction 是多少？
- FSDP 与 DDP 的差别是通信算法不同，还是训练状态的所有权不同？
- All-to-all 为什么天然适合 MoE，而不一定适合普通 dense MLP？

11.3 工程 checklist

原则	动作
先量再切	测 all-reduce / all-gather 的真实带宽，不要拿理论线速代替 step time。
先拓扑后策略	列出 NVLink 域、节点边界、网卡数量，再决定 TP / PP / DP 的 group 大小。
切分轴要一致	确认模型里的每一个参数、gradient、optimizer state 都有明确 owner。
给通信留 overlap	优先把 collective 分块，尽量塞进 backward 和下一层计算之间。
观察长尾	最慢 rank 决定同步 step；检查负载均衡、消息大小和 bucket 边界。
从小组合开始	先让 2 卡或 4 卡跑通，再扩展到 node、pod，避免一次堆叠过多策略。

> **最终心智模型**：把每个并行策略都写成 (sharding axis, replicated state, communication primitive, hardware tier) 四元组。能写出这四项，就已经掌握了 Lecture 7 的主干。

参考来源

主要来源：CS336 lecture_07.py、课程图片 gpu-node-overview.png / ranks.png / data-parallelism.png / tensor-parallelism.png / pipeline-parallelism.png。补充公式：标准 ring all-reduce 通信量、pipeline bubble 近似和 FSDP/ZeRO 的模型状态账本。补充内容是为了建立工程直觉，明确标注为讲义之外的分析。